# Stages, File Formats, and COPY INTO

DATA PIPELINE AUTOMATION IN SNOWFLAKE

**Emily Melhuish**
Technical Curriculum Developer,
Snowflake

datacamp

# Snowflake powers many workloads

**DATA**

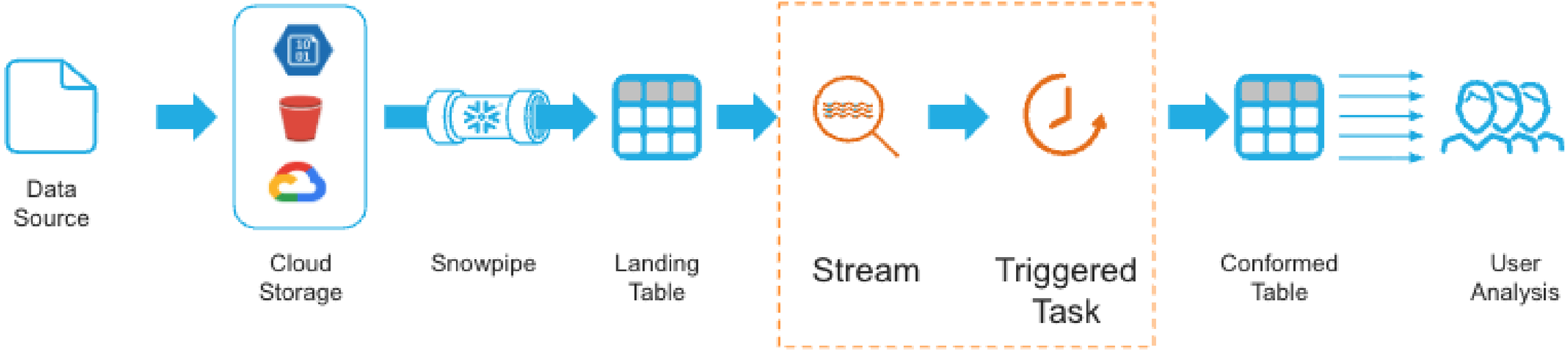
Structured
Semi-structured
Unstructured
Streaming


**OUTCOMES**

Insights
Predictions
Monetization
Data Products

[1] * Snowflake Learning Material

# Example pipeline



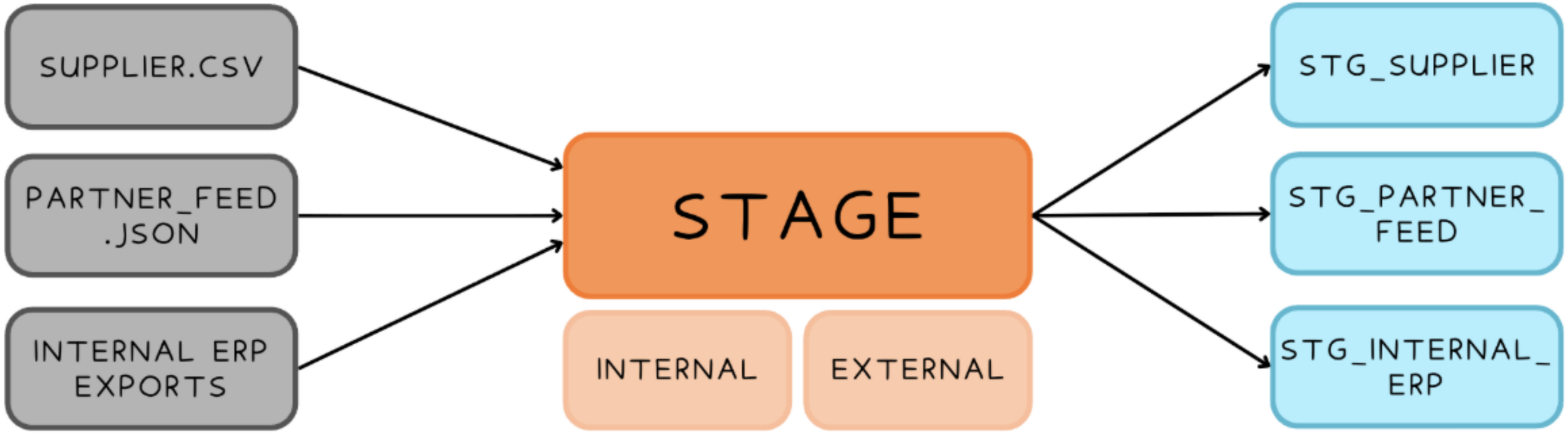

[1] * Snowflake Learning Material

# Meet Harbr


**About Harbr**

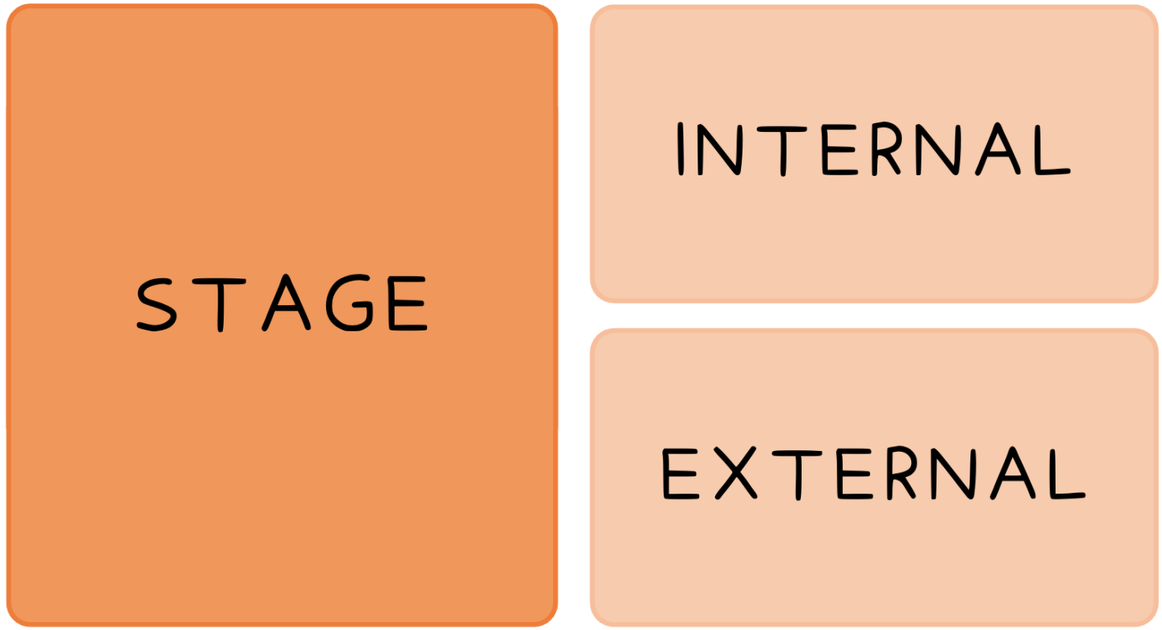
- Global logistics and supply chain platform
- Uses Snowflake to track shipments, warehouse inventory, and delivery performance
- Ingests data from supplier systems across multiple regions

# The Data Loading Problem




Stage = temporary storage location

# Stage Types




**Internal Stages**

- User
  - Personal staging area for a single user
- Table
  - Tied to a specific table
- Named
  - Database object that gets created in the schema

**External Stage**

- Named
  - Points to cloud provider

# Stage Parameters

## Directory Table

```
CREATE STAGE harbr_stage
  DIRECTORY = (ENABLE = TRUE)
```

## Internal Stage Encryption

```
ENCRYPTION = (TYPE = SNOWFLAKE_FULL)
ENCRYPTION = (TYPE = SNOWFLAKE_SSE)
```

## External Stage Encryption

```
ENCRYPTION =
([ TYPE = 'AWS_CSE' ] MASTER_KEY = '')

ENCRYPTION =
([ TYPE = 'AWS_SSE_S3' ])

...
```

# File Formats

- Structured formats: CSV with specific delimiters, header rows, quoting rules
- Semi-structured formats: JSON, Parquet, Avro, ORC and XML.
  - Load directly into a `VARIANT` column
  - Define parsing rules once, reuse across all load operations
- Update the format object once

**SQL**

```
CREATE FILE FORMAT harbr_csv_format
  TYPE = 'CSV'
  FIELD_DELIMITER = ','
  SKIP_HEADER = 1;
```

# COPY INTO

**Load from a named stage using a named file format**

```
COPY INTO logistics.shipments
FROM @harbr_internal_stage/shipments/
FILE_FORMAT = (FORMAT_NAME = 'harbr_csv_format')
```

# Error handling

| ON_ERROR Options | Behavior |
|---|---|
| `ABORT_STATEMENT` | Fail entire load on first error *(default)* |
| `CONTINUE` | Skip bad rows, load everything else |
| `SKIP_FILE` | Skip entire file if it contains any errors |
| `SKIP_FILE_<num>` | Skip file only if errors exceed a specified count |
| `SKIP_FILE_<num>%` | Skip file only if errors exceed a specified % threshold |

# Validation mode and COPY_HISTORY

**Validate without loading** = dry run

```
COPY INTO logistics.shipments
FROM @harbr_internal_stage/shipments/
FILE_FORMAT = (FORMAT_NAME = 'harbr_csv_format')
VALIDATION_MODE = RETURN_ERRORS;
```

**Inspect load history**

```
SELECT * FROM TABLE(INFORMATION_SCHEMA.COPY_HISTORY(
  TABLE_NAME => 'shipments',
  START_TIME => DATEADD('hour', -24,
    CURRENT_TIMESTAMP())));
```

# Let's practice!

DATA PIPELINE AUTOMATION IN SNOWFLAKE

# Snowpipe and Snowpipe Streaming

DATA PIPELINE AUTOMATION IN SNOWFLAKE

**Emily Melhuish**
Technical Curriculum Developer,
Snowflake

datacamp

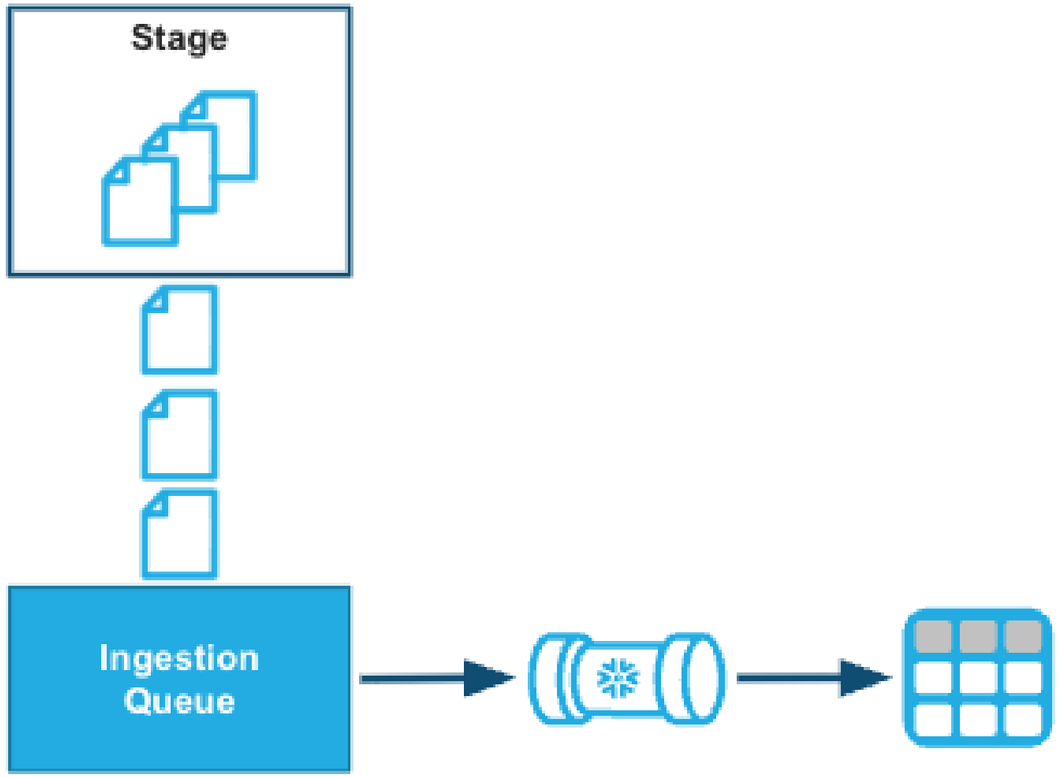
# Snowpipe

**Use Case:**

- Delivery events arrive continuously

**Solution: Snowpipe**

- Snowpipe loads data from files as soon as they are available in a stage.




[1] * Snowflake Learning Material

# The Problem with Batch Loading

- Files arrive in S3 every few minutes throughout the day
- Scheduled `COPY INTO` runs at midnight — 24-hour lag
- Delayed shipments won't appear until the following day.
- Snowpipe closes the gap

```
-- Nightly batch: runs at 00:00, data arrives all day
COPY INTO logistics.delivery_events
FROM @harbr_s3_stage/events/
FILE_FORMAT = (FORMAT_NAME = 'harbr_json_format');
-- A 9am exception won't appear until tomorrow
```

# What is Snowpipe?

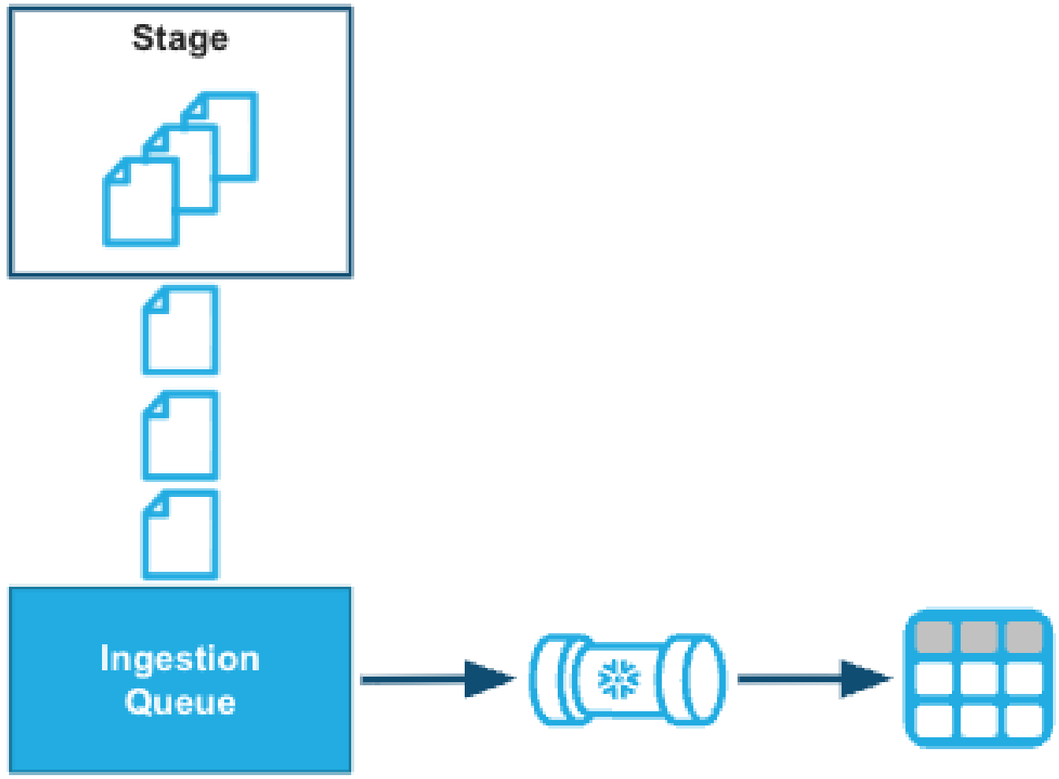
- Wraps a `COPY INTO` statement — same syntax, same file formats
- Triggers automatically as new files arrive in a stage
- Loads in micro-batches, typically within minutes
- **Serverless** — no warehouse to provision

```
CREATE PIPE harbr_events_pipe AS
  COPY INTO logistics.delivery_events
  FROM @harbr_s3_stage/events/
  FILE_FORMAT = (FORMAT_NAME = 'harbr_json_format');
```

# How Snowpipe Works


- `AUTO_INGEST` — event-driven; cloud storage publishes a notification
- Amazon S3 | Azure Event Grid | GCP Pub/Sub
- **REST API trigger** — call `insertFiles` or `insertReport` endpoints directly from orchestration code

# Snowpipe Billing




- Billed based on a fixed credit amount per GB consumed
- Text files: charge based on uncompressed size
- Binary files: charged based on observed size

# Snowpipe Streaming

| | Snowpipe | Snowpipe Streaming |
|---|---|---|
| Trigger | File lands in stage | Row written by application |
| Latency | Minutes | Seconds |
| Use case | File-based event feeds | GPS, IoT, real-time app data |

**Removes the file boundary entirely**

- Rows written directly from the application via the Streaming Ingest SDK
- No files, no stages — latency in **seconds**

```
# Snowpipe Streaming: application writes rows directly
channel = client.openChannel('GPS_CHANNEL', 'LOGISTICS', 'GPS_EVENTS')
```

# Choosing the Right Ingestion Method


| Method | When to use |
|---|---|
| **COPY INTO** | Scheduled batch loads - nightly files, weekly exports, hours of latency acceptable |
| **Snowpipe** | Continuous file arrivals, loads needed within minutes of arrival |
| **Snowpipe Streaming** | Application-generated data - GPS, IoT, financial markets - data available in seconds |

[1] * Snowflake Learning Resource

# Let's practice!

DATA PIPELINE AUTOMATION IN SNOWFLAKE

# Data Unloading and Connectivity

DATA PIPELINE AUTOMATION IN SNOWFLAKE

**Emily Melhuish**
Technical Curriculum Developer,
Snowflake

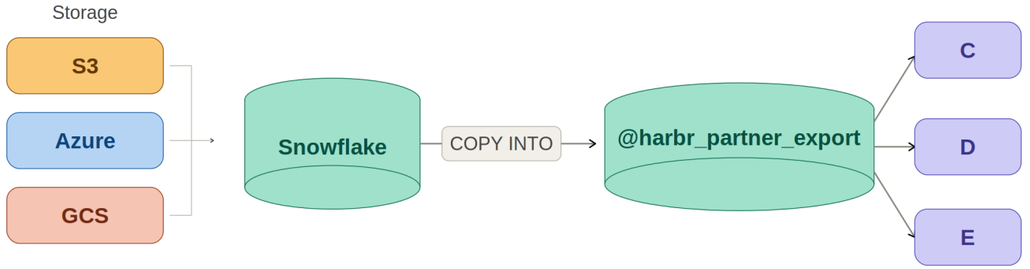
datacamp

# The Unloading Use Case

## Exporting data to cloud storage

- Not all partners have Snowflake access
- `COPY INTO` writes results directly to a stage
- No separate export pipeline needed

```
COPY INTO @harbr_partner_export/
daily_summary/
FROM (SELECT * FROM
logistics.shipment_summary
WHERE export_date =
CURRENT_DATE() - 1);
```



# COPY INTO

```
COPY INTO @harbr_partner_export/shipment_summary/
FROM (
  SELECT shipment_id,
         origin,
         destination,
         delivery_status,
         delivery_time_hours
  FROM logistics.shipments
  WHERE delivery_date = CURRENT_DATE()
)
FILE_FORMAT = (TYPE = 'CSV' HEADER = TRUE)
OVERWRITE = TRUE;
```

# Export File Format Options

| Format | Best For |
|---|---|
| **CSV** | Universal - almost every system can read it; ideal for partner exports |
| **JSON** | Preserves nested structures; useful for semi-structured output consumers |
| **Parquet** | Large analytical datasets; columnar compression = smaller files, faster reads |

**Key unloading options**

```
-- Split large exports across multiple files (bytes)
HEADER = TRUE                -- include column names in first row
OVERWRITE = TRUE             -- replace any existing files at that path
MAX_FILE_SIZE = 104857600  -- 100 MB per output file
```

# The Connectivity Landscape

# Kafka and Spark connectors

### Kafka Connector

- Topics stream directly into Snowflake tables
- No files, no stages — uses Snowpipe Streaming
- Latency measured in seconds

```
# connector.properties (Kafka settings)
snowflake.topic2table.map=events:
delivery_events
snowflake.ingestion.method=
SNOWPIPE_STREAMING
```

### Spark Connector

- Integrates with Spark's DataFrame API
- Spark jobs read from and write to Snowflake
- Handles large-scale transformation workloads

```
// Read from Snowflake into a Spark DataF
val df = spark.read.format("snowflake")
  .options(sfOptions).option("dbtable",
    "shipments").load()
```

# JDBC/ODBC and Partner Integrations

## Universal connectivity

| Integration | Type | Use at Harbr |
|---|---|---|
| JDBC / ODBC | Universal drivers | BI tools (Tableau, Power BI, Looker) - query directly |
| Python Connector | Native Python driver | Data pipelines, scheduled ETL, data science workflows |
| dbt | SQL transformation | Runs models directly in Snowflake compute |
| Fivetran / Airbyte | Managed ingestion | SaaS source systems → Snowflake, no custom code |

# Let's practice!

DATA PIPELINE AUTOMATION IN SNOWFLAKE